

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 5

Выполнил:

Дмитриев Андрей

Группа

К3439

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1. Задача

Выполнить разделение существующего API на микросервисную архитектуру.

Требование: **минимум 3 микросервиса**.

2. Ход работы

2.1 Логическое разбиение системы (модули → сервисы)

Было выполнено выделение отдельных зон ответственности и перенос их в независимые сервисы.

СХЕМА: МИКРОСЕРВИСЫ И ОТВЕТСТВЕННОСТЬ

1. **Main-service**

- Сущности: User, Student, Teacher, Course, Lesson, Group, Schedule, Events, News, Products, Points, Notifications
- База данных: **school_db (PostgreSQL)**
- Порт БД: **5435 (внешний)**
- Причина выделения: основной бизнес-контур системы (пользователи, обучение, расписание, события, магазин, баллы)

2. **Email-service**

- Сущности: EmailLog (логирование отправок), отправка email-уведомлений
- База данных: **email_db (PostgreSQL)**
- Порт БД: **5436 (внешний)**
- Причина выделения: отдельный сервис для асинхронной отправки писем и логирования, изоляция от основной логики

3. **Storage-service (MinIO)**

- Ответственность: файлы и изображения (аватары, материалы уроков, картинки новостей/товаров)
- Хранилище: **S3-совместимое Object Storage (MinIO)**
- Данные: **minio_data (/data)**

- Причина выделения: хранение файлов отдельно от БД, удобное масштабирование и стандартный S3 API
-

2.2 Набор эндпоинтов каждого сервиса

2.2.1 Main-service (порт :8080)

CHEMA: USERS API — /api/users

- POST /api/users/auth — авторизация пользователя → {user, tokens}
- POST /api/users/refresh — обновление токена (Bearer) → {user, tokens}
- POST /api/users/logout — выход (Bearer) → 204
- GET /api/users/me — текущий пользователь (Bearer) → {user}
- POST /api/users/change_password — смена пароля (Bearer) → {user}
- POST /api/users/impersonate — вход под другим пользователем (admin) → {user}
- POST /api/users/ — создать пользователя (Bearer) → 201, {user}
- GET /api/users/ — список пользователей (Bearer) → {users[]}
- GET /api/users/{user_id} — пользователь по ID (Bearer) → {user}
- PUT /api/users/{user_id} — обновить пользователя (Bearer) → {user}
- DELETE /api/users/{user_id} — удалить пользователя (Bearer) → 204
- POST /api/users/reset_password — запрос сброса пароля → 204
- POST /api/users/confirm_reset_password — подтверждение сброса → {user}

CHEMA: STUDENTS API — /api/students

- POST /api/students/ — создать студента (Bearer) → {student}
- GET /api/students/me — текущий студент (Bearer) → {student}
- GET /api/students/ — список студентов (Bearer) → {students[]}
- GET /api/students/{student_id} — студент по ID (Bearer) → {student}
- PUT /api/students/{student_id} — обновить студента (Bearer) → {student}
- DELETE /api/students/{student_id} — удалить студента (Bearer) → 204

CHEMA: TEACHERS API — /api/teachers

- POST /api/teachers/ — создать преподавателя (Bearer) → {teacher}
- GET /api/teachers/me — текущий преподаватель (Bearer) → {teacher}
- GET /api/teachers/ — список преподавателей (Bearer) → {teachers[]}
- GET /api/teachers/{teacher_id} — преподаватель по ID (Bearer) → {teacher}
- PUT /api/teachers/{teacher_id} — обновить преподавателя (Bearer) → {teacher}
- DELETE /api/teachers/{teacher_id} — удалить преподавателя (Bearer) → 204

CHEMA: COURSES API — /api/courses

- POST /api/courses/ — создать курс (teacher) → {course}
- GET /api/courses/ — список курсов (Bearer) → {courses[]}
- GET /api/courses/{course_id} — курс по ID (Bearer) → {course}
- PUT /api/courses/{course_id} — обновить курс (teacher) → {course}
- DELETE /api/courses/{course_id} — удалить курс (teacher) → 204
- GET /api/courses/teacher — курсы преподавателя (teacher) → {courses[]}
- GET /api/courses/student — курсы студента (student) → {courses[]}

CHEMA: LESSONS API — /api/courses/lessons

- POST /api/courses/{course_id}/lessons — создать урок → {lesson}
- GET /api/courses/{course_id}/lessons — список уроков → {lessons[]}
- GET /api/courses/lessons/{lesson_id} — урок по ID → {lesson}
- PUT /api/courses/lessons/{lesson_id} — обновить урок → {lesson}
- DELETE /api/courses/lessons/{lesson_id} — удалить урок → 204

CHEMA: GROUPS API — /api/groups

- POST /api/groups/ — создать группу (Bearer) → 201, {group}
- GET /api/groups/ — список групп → {groups[]}
- GET /api/groups/teacher — группы преподавателя (Bearer) → {groups[]}
- GET /api/groups/student — группы студента (Bearer) → {groups[]}

- GET /api/groups/{group_id} — группа по ID → {group}
- PUT /api/groups/{group_id} — обновить группу (Bearer) → {group}
- DELETE /api/groups/{group_id} — удалить группу (Bearer) → 204
- POST /api/groups/{group_id}/students/ — добавить студентов → {group}
- DELETE /api/groups/{group_id}/students/{student_id} — удалить студента → 204
- POST /api/groups/{group_id}/teacher/ — добавить преподавателя → {group}
- DELETE /api/groups/{group_id}/teacher/{teacher_id} — удалить преподавателя → 204

CHEMA: SCHEDULE API — /api/schedule

- GET /api/schedule/ — получить расписание (Bearer) → {schedule}
- GET /api/schedule/lessons — фильтрованное расписание (Bearer) → {schedule}

CHEMA: EVENTS API — /api/events

- GET /api/events/ — список событий (Bearer) → {events[]}
- POST /api/events/ — создать событие (Bearer) → 201, {event}
- GET /api/events/{event_id} — событие по ID (Bearer) → {event}
- PUT /api/events/{event_id} — обновить событие (Bearer) → {event}
- DELETE /api/events/{event_id} — удалить событие (Bearer) → 204
- POST /api/events/{event_id}/groups — назначить событие группе → {event}
- POST /api/events/{event_id}/users — назначить событие пользователям → {event}
- GET /api/events/{event_id}/users — событие с пользователями → {event}

CHEMA: NEWS API — /api/news

- POST /api/news/ — создать новость → 201, {news}
- GET /api/news/ — список новостей → {news[]}
- GET /api/news/{news_id} — новость по ID → {news}
- PUT /api/news/{news_id} — обновить новость → {news}
- DELETE /api/news/{news_id} — удалить новость → 204

СХЕМА: PRODUCTS API — /api/products

- POST /api/products/ — создать товар (Bearer) → {product}
- GET /api/products/ — список товаров (Bearer) → {products[]}
- GET /api/products/available — доступные товары (Bearer) → {products[]}
- GET /api/products/not-available — недоступные товары (Bearer) → {products[]}
- GET /api/products/{product_id} — товар по ID (Bearer) → {product}
- PUT /api/products/{product_id} — обновить товар (Bearer) → {product}
- DELETE /api/products/{product_id} — удалить товар (Bearer) → 204

СХЕМА: POINTS HISTORY API — /api/points/history

- POST /api/points/history/ — создать запись (Bearer) → 201, {history}
- GET /api/points/history/student — история студента (Bearer) → {history[]}

2.2.2 Email-service (порт :9090)

СХЕМА: EMAILS API — /api/emails

- POST /api/emails/send/ — отправка email-уведомления (Bearer) → {email_log}

2.2.3 Storage-service (MinIO, порты :9000 и :9001)

СХЕМА: S3-ОПЕРАЦИИ (MinIO API)

- PUT — загрузка файла в bucket
- GET — скачивание файла из bucket
- DELETE — удаление файла из bucket
- LIST — просмотр списка объектов в bucket

Дополнительно:

- Console UI (порт :9001) — веб-интерфейс управления MinIO.

2.3 Сетевое взаимодействие и авторизация

- Формат обмена данными: **JSON**.
- Авторизация: **JWT** (access_token в cookies или через Bearer header).
- Сервисы работают в общей Docker-сети, взаимодействуют по DNS-именам контейнеров внутри сети.

3. Архитектура Docker-сети

СХЕМА: DOCKER NETWORK (shool_site-network)

Внутри сети размещены:

- БД основного сервиса (PostgreSQL)
 - БД email-сервиса (PostgreSQL)
 - MinIO (S3 storage)
 - main-app (FastAPI)
 - email-app (FastAPI)
- Фронтенд (React) обращается к API через опубликованные порты.

ТЕКСТОВАЯ СХЕМА:

- main-app (FastAPI) :8080
↔ db (PostgreSQL) :5435 (наружу), 5432 (внутри)
- email-app (FastAPI) :9090
↔ email_db (PostgreSQL) :5436 (наружу), 5432 (внутри)
- main-app / frontend
↔ minio (S3) :9000
↔ minio console :9001

4. Порты сервисов (схема)

СХЕМА: ПОРТЫ

- main-app: внутренний 8080 → внешний 8080 (основной API)
- email-app: внутренний 9090 → внешний 9090 (email-уведомления)
- db: внутренний 5432 → внешний 5435 (БД main-service)

- email_db: внутренний 5432 → внешний 5436 (БД email-service)
- minio: внутренний 9000/9001 → внешний 9000/9001 (S3 + консоль)

5. Конфигурация docker-compose.yml (фрагмент)

services:

db:

image: postgres:17-bookworm

container_name: school_stie_db

ports:

- "5435:5432"

networks:

- school_site-network

volumes:

- school_site_data:/var/lib/postgresql/data

email_db:

image: postgres:17-bookworm

container_name: email_db

ports:

- "5436:5432"

networks:

- school_site-network

volumes:

- email_data:/var/lib/postgresql/data

main-app:

build:

dockerfile: ./docker/backend/Dockerfile

container_name: school_stie_backend

ports:

- "8080:8080"

depends_on:

- db

networks:

- shool_site-network

email-app:

build:

dockerfile: ./docker/email/Dockerfile

container_name: school_stie_email_notifier

ports:

- "9090:9090"

depends_on:

- email_db

networks:

- shool_site-network

minio:

image: minio/minio:latest

ports:

- "9000:9000"

- "9001:9001"

command: server --console-address ":9001" /data

networks:

- shool_site-network

6. Технологический стек (схема)

СХЕМА: STACK

- Backend: FastAPI (Python)
- ORM: SQLAlchemy
- Database: PostgreSQL 17
- Object Storage: MinIO (S3-compatible)
- Containerization: Docker, Docker Compose
- Authentication: JWT (access_token + refresh_token)

7. Вывод

В рамках лабораторной работы API системы IT School было разделено на **три микросервиса**:

1. **Main-service** — основной бизнес-сервис: пользователи, курсы, уроки, группы, расписание, события, новости, товары, баллы и уведомления.
2. **Email-service** — сервис отправки email-уведомлений и логирования писем, с отдельной базой данных.
3. **Storage-service (MinIO)** — S3-совместимое хранилище файлов и изображений.

Каждый сервис:

- имеет отдельную зону ответственности;
- разворачивается в отдельном контейнере;
- использует собственное хранилище данных (БД или object storage);
- может обновляться и масштабироваться независимо.

Преимущества полученной архитектуры:

- масштабируемость (каждый сервис масштабируется отдельно);
- отказоустойчивость (сбой одного компонента меньше влияет на остальные);
- независимое развёртывание (обновление сервисов без остановки всей системы);
- изоляция данных (раздельные базы для main и email).